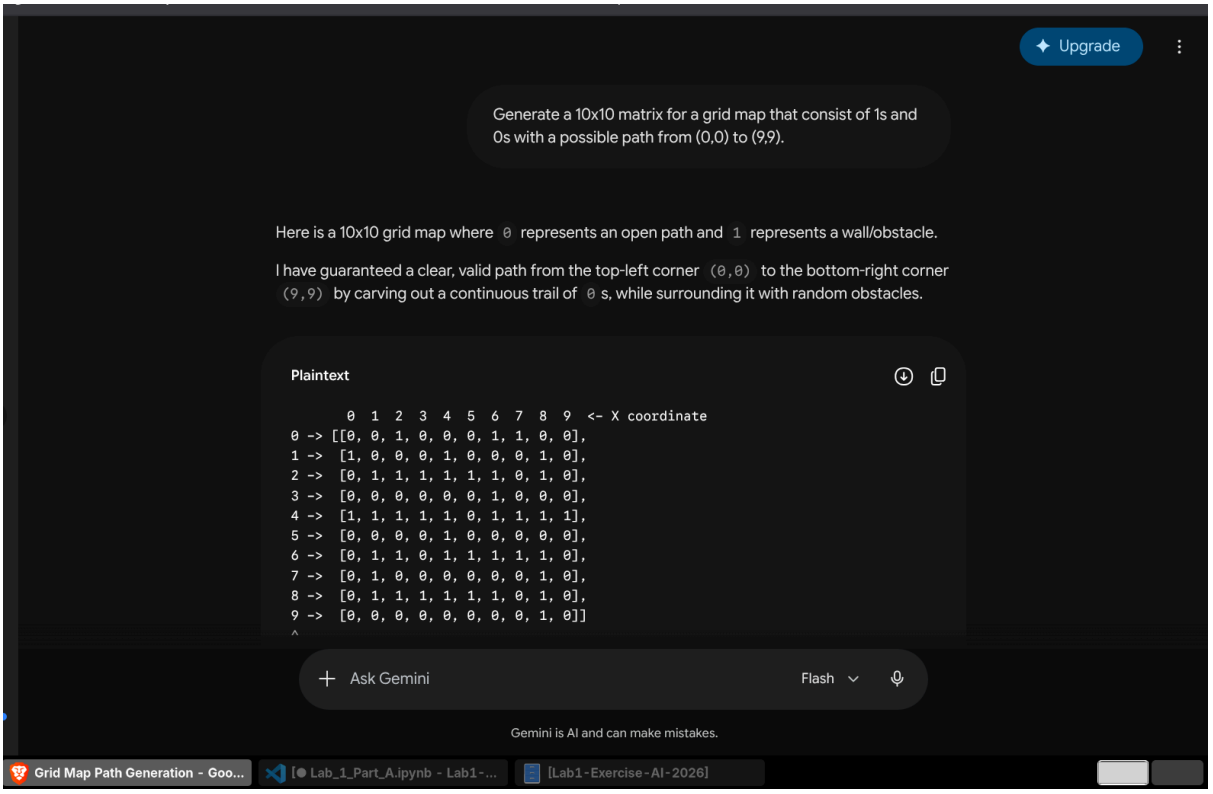
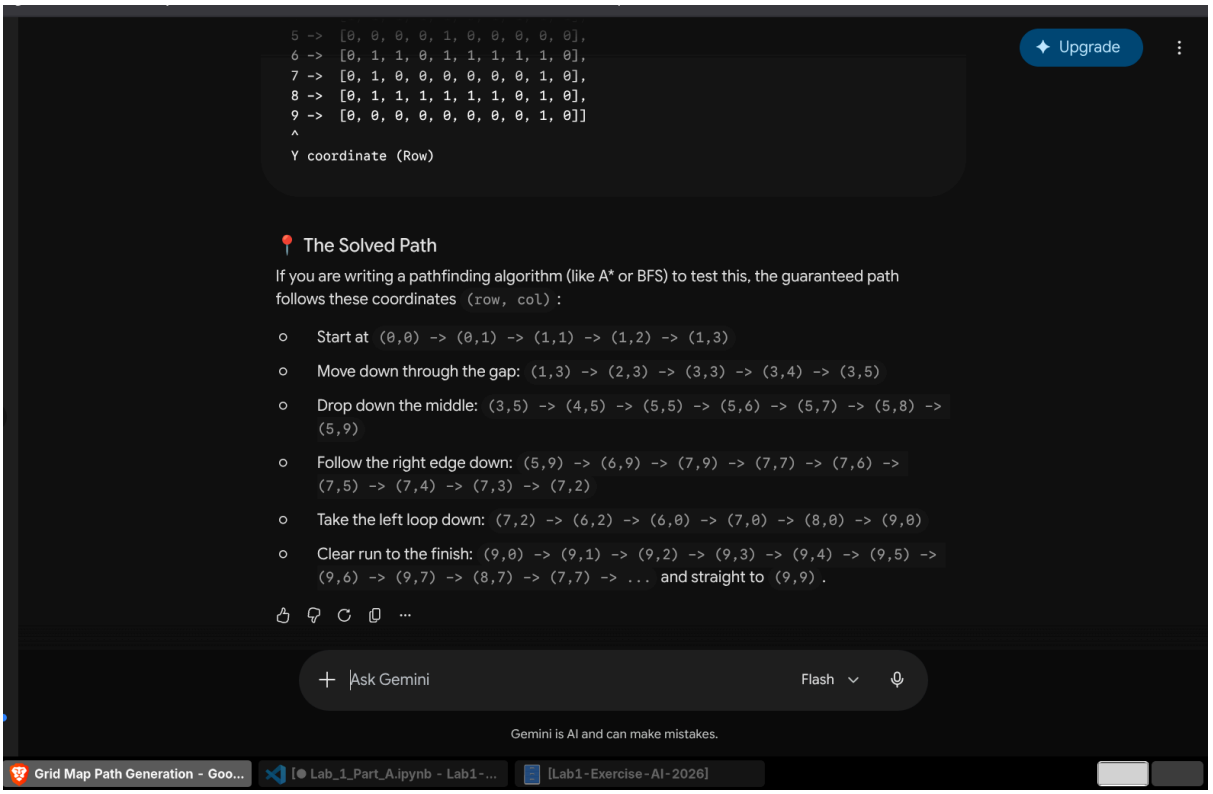
Below are screenshots of my interaction with Gemini AI while I was working on the lab 1 assignment.

Disclaimer: This file doesn’t not indicate my use of AI to complete this assignment but it serves as a commitment to Ashesi values and to the integrity of the contract signed in class earlier this semester.


Upgrade
Generate a 10x10 matrix for a grid map that consist of 1s and 0s with a possible path from (0,0) to (9,9).
Here is a 10x10 grid map where 0 represents an open path and 1 represents a wall/obstacle.
I have guaranteed a clear, valid path from the top-left corner (0,0) to the bottom-right corner (9,9) by carving out a continuous trail of 0 s, while surrounding it with random obstacles.
Plaintext
0 1 2 3 4 5 6 7 8 9 <- X coordinate
0 -> [[0, 0, 1, 0, 0, 0, 1, 1, 0, 0],
1 -> [1, 0, 0, 0, 1, 0, 0, 0, 1, 0],
2 -> [0, 1, 1, 1, 1, 1, 1, 0, 1, 0],
3 -> [0, 0, 0, 0, 0, 0, 1, 0, 0, 0],
4 -> [1, 1, 1, 1, 1, 0, 1, 1, 1, 1],
5 -> [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
6 -> [0, 1, 1, 0, 1, 1, 1, 1, 1, 0],
7 -> [0, 1, 0, 0, 0, 0, 0, 0, 1, 0],
8 -> [0, 1, 1, 1, 1, 1, 1, 1, 0, 0],
9 -> [0, 0, 0, 0, 0, 0, 0, 0, 1, 0]]
^
Ask Gemini
Flash
Gemini is AI and can make mistakes.
Grid Map Path Generation - Goo...
[Lab_1_Part_A.ipynb - Lab1-...
[Lab1-Exercise-AI-2026]


5 -> [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
6 -> [0, 1, 1, 0, 1, 1, 1, 1, 1, 0],
7 -> [0, 1, 0, 0, 0, 0, 0, 0, 1, 0],
8 -> [0, 1, 1, 1, 1, 1, 1, 1, 0, 0],
9 -> [0, 0, 0, 0, 0, 0, 0, 0, 1, 0]]
^
Y coordinate (Row)
Upgrade
📍 The Solved Path
If you are writing a pathfinding algorithm (like A* or BFS) to test this, the guaranteed path follows these coordinates (row, col):
Start at (0,0) -> (0,1) -> (1,1) -> (1,2) -> (1,3)
Move down through the gap: (1,3) -> (2,3) -> (3,3) -> (3,4) -> (3,5)
Drop down the middle: (3,5) -> (4,5) -> (5,5) -> (5,6) -> (5,7) -> (5,8) -> (5,9)
Follow the right edge down: (5,9) -> (6,9) -> (7,9) -> (7,7) -> (7,6) -> (7,5) -> (7,4) -> (7,3) -> (7,2)
Take the left loop down: (7,2) -> (6,2) -> (6,0) -> (7,0) -> (8,0) -> (9,0)
Clear run to the finish: (9,0) -> (9,1) -> (9,2) -> (9,3) -> (9,4) -> (9,5) -> (9,6) -> (9,7) -> (8,7) -> (7,7) -> ... and straight to (9,9).
Ask Gemini
Flash
Gemini is AI and can make mistakes.
Grid Map Path Generation - Goo...
[Lab_1_Part_A.ipynb - Lab1-...
[Lab1-Exercise-AI-2026]

Upgrade

Generate a 10x10 matrix for a grid map that consist of 1s and 0s with a possible path from (0,0) to (9,9).

Here is a 10x10 grid map where `0` represents an open path and `1` represents a wall/obstacle.

I have guaranteed a clear, valid path from the top-left corner `(0,0)` to the bottom-right corner `(9,9)` by carving out a continuous trail of `0`s, while surrounding it with random obstacles.

Plaintext

```
      0  1  2  3  4  5  6  7  8  9  <- X coordinate
0 -> [[0, 0, 1, 0, 0, 0, 1, 1, 0, 0],
1 ->  [1, 0, 0, 0, 1, 0, 0, 0, 1, 0],
2 ->  [0, 1, 1, 1, 1, 1, 1, 0, 1, 0],
3 ->  [0, 0, 0, 0, 0, 0, 1, 0, 0, 0],
4 ->  [1, 1, 1, 1, 1, 0, 1, 1, 1, 1],
5 ->  [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
6 ->  [0, 1, 1, 0, 1, 1, 1, 1, 1, 0],
7 ->  [0, 1, 0, 0, 0, 0, 0, 0, 1, 0],
8 ->  [0, 1, 1, 1, 1, 1, 1, 0, 1, 0],
9 ->  [0, 0, 0, 0, 0, 0, 0, 0, 1, 0]]
^
```

Ask Gemini Flash

Gemini is AI and can make mistakes.

Grid Map Path Generation - Goo... [● Lab_1_Part_A.ipynb - Lab1-... [Lab1-Exercise-AI-2026]

Upgrade

List the common OOP in python concepts that are using in the search algorithms formulation, for both the informed and uninformed search.

In Python, formulating both uninformed search algorithms (like BFS and DFS) and informed search algorithms (like A* and Greedy Best-First Search) relies heavily on Object-Oriented Programming (OOP). OOP allows us to model the abstract components of a search problem—such as the state space, the graph structure, and the frontier—into clean, reusable code.

Here are the common OOP concepts used to build these search algorithms:

1. Classes and Objects (Encapsulation)

The foundation of any OOP program is bundling data and the methods that operate on that data into a single unit. In search algorithms, we use classes to represent the core entities of the problem.

- `Node` **Class:** Represents a state in the search tree. It encapsulates the current state, a pointer to its `parent` node (to reconstruct the path), the `action` taken to get there, and the path cost ($g(n)$). For informed searches, it also stores the heuristic value ($h(n)$) or total evaluation cost ($f(n)$).
- `Problem` / `Graph` **Class:** Encapsulates the rules of the world. It contains the `initial_state`, the `goal_test()` method, and a `get_successors()` method to

Ask Gemini Flash

Gemini is AI and can make mistakes.

Grid Map Path Generation - Goo... [● Lab_1_Part_A.ipynb - Lab1-... [Lab1-Exercise-AI-2026]

- `Problem` / `Graph` Class: Encapsulates the rules of the world. It contains the `initial_state`, the `goal_test()` method, and a `get_successors()` method to generate valid moves from a given state.

Upgrade

Python

```
class Node:
    def __init__(self, state, parent=None, action=None, path_cost=0, heuristic=0):
        self.state = state
        self.parent = parent
        self.action = action
        self.path_cost = path_cost  # Used in Uninformed (UCS) & Informed (A*)
        self.heuristic = heuristic  # Used in Informed Search (A*, Greedy)
        self.f_cost = path_cost + heuristic
```

2. Polymorphism (Duck Typing)

Polymorphism allows different search algorithms to use the exact same framework, changing only how the **frontier** (the collection of nodes waiting to be explored) behaves.

In Python, this is achieved through "duck typing." As long as the frontier object has `.append()` (or `.push()`) and `.pop()` methods, the main search loop doesn't care how it works under the hood.

- **Uninformed Search (BFS):** Uses a `Queue` (FIFO - First In, First Out). `pop()` removes the oldest node.
- **Uninformed Search (DFS):** Uses a `Stack` (LIFO - Last In, First Out). `pop()` removes the newest node.
- *Informed Search (A / Greedy):** Uses a `PriorityQueue`. `pop()` always removes the node with the lowest $f(n)$ or $h(n)$ value.

By swapping the frontier object, the exact same `search(problem, frontier)` function can run BFS, DFS, or A*.

3. Operator Overloading (Magic Methods)

For informed search algorithms, the frontier needs to sort nodes based on their costs. Python handles this beautifully using **operator overloading** via "dunder" (double underscore) methods.

To make the `PriorityQueue` understand how to compare two `Node` objects, you override the "less than" (`__lt__`) operator inside the `Node` class.

Python

```
    def __lt__(self, other):
        # Crucial for Informed Search (A*) to sort nodes in the Priority Queue
        return self.f_cost < other.f_cost
```

Ask Gemini Flash

Gemini is AI and can make mistakes.

Grid Map Path Generation - Goo... [● Lab_1_Part_A.ipynb - Lab1-... [Lab1-Exercise-AI-2026]

```
return self.f_cost < other.f_cost
```

Upgrade

Similarly, `__eq__` (equal to) and `__hash__` are often overloaded so that nodes can be easily checked against a `set` of already explored states (`if current_node in explored:`).

4. Abstraction

Abstraction hides complex implementation details and only shows the essential features. In search formulations, an **Abstract Base Class (ABC)** is often used to define a template for a search problem.

If you are building a generic search solver, you create an abstract `Problem` class. Anyone who wants to use your search algorithm must inherit from this class and implement the specific details for their game or map.

Python

```python
from abc import ABC, abstractmethod

class SearchProblem(ABC):
    @abstractmethod
    def actions(self, state):
        """Returns valid actions from the given state."""
        pass

    @abstractmethod
```

Ask Gemini Flash

Gemini is AI and can make mistakes.

Grid Map Path Generation - Goo... [● Lab_1_Part_A.ipynb - Lab1-... [Lab1-Exercise-AI-2026]

Upgrade

Summary: How Informed vs. Uninformed Differ in OOP

While both paradigms use all of these concepts, the way they leverage them highlights their differences:

OOP Feature	Uninformed Search (BFS, DFS, UCS)	Informed Search (A*, Greedy Best-First)
Class Attributes	`Node` properties are limited to `state` , `parent` , and basic `path_cost` .	`Node` properties must include `heuristic` and evaluation functions ($f = g + h$).
Operator Overloading	Mostly relies on `__eq__` and `__hash__` to track visited states in a set.	Heavily relies on `__lt__` so the priority queue can automatically sort nodes by priority.
Polymorphism	The frontier is a standard sequence container (deque/list acting as Stack or Queue).	The frontier is a specialized container paired with Python's `heapq` module.

...

Ask Gemini Flash

Gemini is AI and can make mistakes.

Grid Map Path Generation - Goo... [● Lab_1_Part_A.ipynb - Lab1-... [Lab1-Exercise-AI-2026]

differences:

Upgrade

OOP Feature	Uninformed Search (BFS, DFS, UCS)	Informed Search (A*, Greedy Best-First)
Class Attributes	`Node` properties are limited to `state`, `parent`, and basic `path_cost`.	`Node` properties must include `heuristic` and evaluation functions ($f = g + h$).
Operator Overloading	Mostly relies on `__eq__` and `__hash__` to track visited states in a set.	Heavily relies on `__lt__` so the priority queue can automatically sort nodes by priority.
Polymorphism	The frontier is a standard sequence container (deque/list acting as Stack or Queue).	The frontier is a specialized container paired with Python's `heapq` module.

...

Are you implementing a specific algorithm right now, or looking to see how these classes fit into a main search loop?

Ask Gemini Flash

Gemini is AI and can make mistakes.

Grid Map Path Generation - Goo... [● Lab_1_Part_A.ipynb - Lab1-... [Lab1-Exercise-AI-2026]

Upgrade

Generate for me the common data structures that are used in search algorithms, this should be in python and also expand more on the nodes, queue, dequeue and the FIFO and LIPO

To implement search algorithms effectively in Python, you need a solid grasp of specific data structures. These structures manage the **frontier** (the nodes waiting to be explored) and the **explored set** (the states we've already visited). How these structures store and release data dictates the entire search strategy.

Here is a breakdown of the essential data structures, with a deep dive into nodes, queues, deques, and ordering principles like FIFO and LIFO.

1. The Building Block: The Node

In search algorithms, a **Node** is a custom data structure (usually a class) that acts as a wrapper around a state. It is crucial to understand the difference between a *State* and a *Node*:

- **State:** A representation of a physical configuration of the world (e.g., the coordinate `(3, 5)` on your grid map).
- **Node:** A bookkeeping structure that contains the state, but also includes the metadata

Ask Gemini Flash

Gemini is AI and can make mistakes.

Grid Map Path Generation - Goo... [● Lab_1_Part_A.ipynb - Lab1-... [Lab1-Exercise-AI-2026]

- **Node:** A bookkeeping structure that contains the state, but also includes the metadata needed to construct the search tree.

Anatomy of a Node Class

Python

```
class Node:
    def __init__(self, state, parent=None, action=None, path_cost=0):
        self.state = state        # The actual position/configuration (e.g., (x, y))
        self.parent = parent      # The Node we came from (allows path reconstructio
        self.action = action      # The move that got us here (e.g., "UP", "LEFT")
        self.path_cost = path_cost # Cumulative cost g(n) from start to this node
```

Without the `parent` pointer in the Node structure, an algorithm might find the goal state, but it would have no memory of the exact sequence of steps taken to get there.

2. Queue, Deque, FIFO, and LIFO

The frontier requires a linear data structure where elements can be added and removed. Depending on *how* elements are removed, the algorithm behaves entirely differently.

Ask Gemini Flash

Gemini is AI and can make mistakes.

Grid Map Path Generation - Goo... [● Lab_1_Part_A.ipynb - Lab1-... [Lab1-Exercise-AI-2026]

2. Queue, Deque, FIFO, and LIFO

The frontier requires a linear data structure where elements can be added and removed. Depending on *how* elements are removed, the algorithm behaves entirely differently.

Queue (The Abstract Concept)

A queue is a general term for a collection of elements kept in a specific order. The two primary types of queues used in uninformed search are defined by their access constraints: FIFO and LIFO.

FIFO (First-In, First-Out) → Used in Breadth-First Search (BFS)

- **How it works:** Like a real-world line at a grocery store. The first item added to the queue is the first one to be removed.
- **Search Behavior:** Because it explores all nodes at the current depth level before moving deeper, FIFO guarantees finding the shallowest path to a goal.

LIFO (Last-In, First-Out) → Used in Depth-First Search (DFS)

- **How it works:** Like a stack of dinner plates. The last item you place on top of the stack is the first one you take off.
- *Note:* While the user mentioned "LIPO," the correct technical acronym is **LIFO** (Last-In, First-Out).

Ask Gemini Flash

Gemini is AI and can make mistakes.

Grid Map Path Generation - Goo... [● Lab_1_Part_A.ipynb - Lab1-... [Lab1-Exercise-AI-2026]

First-Out).

- **Search Behavior:** It forces the algorithm to plunge straight down a single path to its maximum depth before backtracking.

The Solution in Python: `collections.deque`

While you can technically use a standard Python list as a queue (using `.append()` and `.pop(0)`), **this is highly inefficient**. Removing the first element of a standard list takes $O(n)$ time because Python has to shift all remaining elements in memory.

To solve this, Python provides `collections.deque` (Double-Ended Queue). A deque allows you to add and remove elements from *both* ends in optimal $O(1)$ constant time.

Python

```
from collections import deque

# --- FIFO Queue Implementation (for BFS) ---
fifo_frontier = deque()
fifo_frontier.append(node_A)  # Add to the right
fifo_frontier.append(node_B)
# To maintain FIFO, we pop from the LEFT (the oldest element)
next_node = fifo_frontier.popleft()  # Returns node_A

# --- LIFO Stack Implementation (for DFS) ---
lifo_frontier = deque()
```

Ask Gemini Flash

Gemini is AI and can make mistakes.

Grid Map Path Generation - Goo... [● Lab_1_Part_A.ipynb - Lab1-... [Lab1-Exercise-AI-2026]

Upgrade

```
# --- LIFO Stack Implementation (for DFS) ---
lifo_frontier = deque()
lifo_frontier.append(node_A)  # Add to the right
lifo_frontier.append(node_B)
# To maintain LIFO, we pop from the RIGHT (the newest element)
next_node = lifo_frontier.pop()      # Returns node_B
```

3. Other Essential Search Data Structures

Beyond the standard queue/deque, search algorithms require a couple of other specialized structures to run efficiently:

Priority Queue (`heapq`) → Used in Uniform Cost (UCS) and Informed Search (A*)

In algorithms where paths have weights (like a grid map with terrain costs) or use heuristics, FIFO/LIFO isn't enough. We need to pop the node with the **lowest cost**.

- Python's `heapq` module converts a standard list into a binary heap.
- It allows you to push elements and pop the smallest element in $O(\log n)$ time.

Python

```
import heapq
```

Ask Gemini Flash

Gemini is AI and can make mistakes.

Grid Map Path Generation - Goo... [● Lab_1_Part_A.ipynb - Lab1-... [Lab1-Exercise-AI-2026]

- It allows you to push elements and pop the smallest element in $O(\log n)$ time.

Upgrade

Python

```python
import heapq

# The frontier is initialized as a list
priority_frontier = []

# Elements are pushed as tuples: (priority_value, object)
heapq.heappush(priority_frontier, (5, "Node With Higher Cost"))
heapq.heappush(priority_frontier, (1, "Node With Lowest Cost"))

# Always pops the element with the lowest priority value
lowest_cost_node = heapq.heappop(priority_frontier)  # Returns (1, "Node With Lowest
```

Hash Set (`set`) → Used for the Explored/Visited Set

To prevent algorithms from spinning in infinite loops (e.g., moving back and forth between two grid squares), you must keep track of states you have already evaluated.

- Python's `set()` uses a hash table under the hood.
- Checking if a state has been visited (`if state in explored:`) takes $O(1)$ time on average, making it incredibly fast.

Ask Gemini Flash

Gemini is AI and can make mistakes.

Grid Map Path Generation - Goo... [● Lab_1_Part_A.ipynb - Lab1-... [Lab1-Exercise-AI-2026]

- Python's `set()` uses a hash table under the hood.
- Checking if a state has been visited (`if state in explored:`) takes $O(1)$ time on average, making it incredibly fast.

Upgrade

Quick Reference Summary

Data Structure	Python Implementation	Ordering Principle	Used In
Stack	`collections.deque` paired with `.pop()`	**LIFO** (Last-In, First-Out)	Depth-First Se (DFS)
Queue	`collections.deque` paired with `.popleft()`	**FIFO** (First-In, First-Out)	Breadth-First (BFS)
Priority Queue	`heapq` module functions on a standard `list`	**Lowest Cost First** (Sorted by $f(n)$ or $g(n)$)	Uniform Cost A*, Greedy Be
Set	`set()`	**Unordered**, unique elements	Explored/Visit tracking

Ask Gemini Flash

Gemini is AI and can make mistakes.

Grid Map Path Generation - Goo... [● Lab_1_Part_A.ipynb - Lab1-... [Lab1-Exercise-AI-2026]